D7 – Learning Integration Assessment: Reflection

Project: Boréal Marché – Customer Intelligence Platform
Student: Hossam Siyam
Course: 420-977-VA – AI Development Project
Date: September 2026

---

1. The Hardest Technical Problem I Faced

The hardest technical problem I faced was **deploying the FastAPI application to Render.com**. The API worked perfectly locally and inside a Docker container, but once deployed to Render, the root URL `/` worked while all other endpoints (`/health`, `/docs`, `/info`) returned a plain-text "Not Found" error. The Render logs showed no incoming requests for those paths, which meant the requests were being intercepted by Render's edge router before reaching my application.

At first, I thought it was a code issue — I tried renaming endpoints, adding `HEAD` support, and changing the port. But nothing worked. After hours of debugging, I realized the problem was a **typo in the URL**: I was testing `capstone-bkq2.onrender.com` (with a "Q") instead of the correct `capstone-bkg2.onrender.com` (with a "G"). The URL I had copied from the Render dashboard was misread. Once I tested the correct URL, all endpoints worked immediately.

This problem taught me to **always double-check the exact URL** and to use `curl` to test endpoints rather than relying solely on the browser, which can cache errors.

---

2. How I Solved It

I solved it by:

1. Using `curl` to get the raw HTTP response (which showed `x-render-routing: no-server`).
2. Checking the Render Events page, where I noticed the correct URL had a "G" instead of a "Q".
3. Testing the correct URL and confirming everything worked.
4. Updating the code and documentation to use the correct URL.

---

3. What I Would Do Differently

If I were to do this project again, I would:

- **Use a URL shortener or a custom domain** from the start to avoid typo issues.
- **Test with `curl` before testing in the browser** to avoid caching.
- **Keep a log of all URLs and credentials** in a single place.
- **Set up monitoring earlier** so that I could see when the API was actually receiving requests.
- **Document the deployment process step by step** so that I could easily reproduce it.

---

4. One Thing I Got Wrong and Later Corrected

I initially used a **random stratified split** for the train/test data without enforcing a temporal split. This caused **data leakage**: features were computed using the full dataset, including the label window, which inflated the model's performance to a perfect PR-AUC of 1.0. I later corrected this by:

- Splitting the data into observation and label windows **before** computing features.
- Ensuring that features were only computed from data before the cutoff date.
- Re-running the model and getting realistic results (PR-AUC ~0.78).

This mistake taught me the importance of **temporal integrity** in machine learning — features must never include future information.

---

5. Key Takeaways

- **Deployment is not the end** – monitoring and governance are equally important.
- **Data leakage can be subtle** – always check that features are computed only from past data.
- **A working system beats a perfect model** – deployment and documentation matter more than a 0.01 improvement in accuracy.
- **Always verify URLs and endpoints** – a small typo can waste hours.

End of Reflection